

Algoritmo de búsqueda binaria

Inicialización de la búsqueda

La implementación comienza creando tres vectores.

```
my $beta =  
  nd->ones([$n_samples,1]);  
  
my $beta_min =  
  nd->ones([$n_samples,1]) * -1e20;  
  
my $beta_max =  
  nd->ones([$n_samples,1]) * 1e20;
```

Cada observación posee su propio valor de

$$\beta_i.$$

Por ejemplo,

$$\beta = \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix}$$

Inicialmente,

todos los puntos comienzan con el mismo valor.

También se almacenan los límites inferior y superior.

$$\beta_{\min} = \begin{bmatrix} -\infty \\ -\infty \\ -\infty \\ -\infty \end{bmatrix}$$
$$\beta_{\max} = \begin{bmatrix} +\infty \\ +\infty \\ +\infty \\ +\infty \end{bmatrix}$$

Cada observación evolucionará de forma independiente durante la búsqueda.

Una iteración de la búsqueda binaria

Supongamos que ya calculamos la entropía para cada observación.

El algoritmo obtiene

$$\Delta H = H(P) - \log(\text{Perplexity})$$

Por ejemplo,

$$\Delta H = \begin{bmatrix} 0.31 \\ -0.18 \\ 0.02 \\ -0.41 \end{bmatrix}$$

Interpretación.

ΔH	Acción
Positivo	Disminuir la entropía
Negativo	Aumentar la entropía

Recordemos que

- aumentar

$$\beta$$

produce una distribución más concentrada;

- disminuir

$$\beta$$

produce una distribución más uniforme.

Por lo tanto,

- si

$$\Delta H > 0$$

debemos **aumentar**

$$\beta;$$

- si

$$\Delta H < 0$$

debemos **disminuir**

$$\beta.$$

Este es exactamente el principio de funcionamiento de la búsqueda binaria.

Implementación vectorizada de la decisión

Una implementación clásica utilizaría un bucle similar al siguiente.

si $\Delta H > 0$

 aumentar β

si $\Delta H < 0$

 disminuir β

Sin embargo,

t-SNE realiza esta operación para miles de observaciones simultáneamente.

La implementación crea una máscara lógica.

```
my $is_greater =
  ($entropy_diff > 0.0)
  ->astype('float64');
```

Por ejemplo,

$$\Delta H = \begin{bmatrix} 0.31 \\ -0.18 \\ 0.02 \\ -0.41 \end{bmatrix}$$

produce

$$is_greater = \begin{bmatrix} 1 \\ 0 \\ 1 \\ 0 \end{bmatrix}$$

La máscara complementaria se obtiene mediante

```
my $is_less =
  nd->ones(...)
  -
  $is_greater;
```

obteniendo

$$is_less = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 1 \end{bmatrix}$$

Estas máscaras permiten seleccionar diferentes operaciones para cada observación mediante álgebra matricial, sin utilizar un solo `if`.

En las siguientes diapositivas veremos cómo estas máscaras actualizan automáticamente los límites

$$\beta_{\min} \quad \text{y} \quad \beta_{\max},$$

completando la búsqueda binaria de forma totalmente vectorizada.

Actualización de los límites de la búsqueda

En una búsqueda binaria clásica mantenemos dos límites:

- límite inferior

$$\beta_{\min}$$

- límite superior

$$\beta_{\max}$$

Después de calcular

$$\Delta H = H(P) - \log(\text{Perplexity}),$$

debemos actualizar dichos límites.

Si

$$\Delta H > 0,$$

la entropía es demasiado grande.

Debemos aumentar

$$\beta.$$

Por lo tanto,

el valor actual pasa a convertirse en el nuevo límite inferior.

$$\boxed{\beta_{\min} \leftarrow \beta}$$

Si

$$\Delta H < 0,$$

la entropía es demasiado pequeña.

Debemos disminuir

$$\beta.$$

En consecuencia,

el valor actual pasa a convertirse en el nuevo límite superior.

$$\beta_{\max} \leftarrow \beta$$

Esta es exactamente la lógica implementada por el algoritmo.

Actualización vectorizada de $\beta_{\min}$

La implementación utiliza la instrucción

```
$beta_min =  
    ($is_greater * $beta)  
    + ($is_less * $beta_min);
```

A primera vista puede parecer complicada,

pero únicamente representa una selección entre dos valores.

Supongamos

$$is_greater = \begin{bmatrix} 1 \\ 0 \\ 1 \\ 0 \end{bmatrix}$$

$$is_less = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 1 \end{bmatrix}$$

Además,

$$\beta = \begin{bmatrix} 1.0 \\ 1.0 \\ 1.0 \\ 1.0 \end{bmatrix}$$

$$\beta_{\min} = \begin{bmatrix} 0.50 \\ 0.25 \\ 0.80 \\ 0.40 \end{bmatrix}$$

Calculamos

$$is_greater \cdot \beta = \begin{bmatrix} 1.0 \\ 0 \\ 1.0 \\ 0 \end{bmatrix}$$

y

$$is_less \cdot \beta_{\min} = \begin{bmatrix} 0 \\ 0.25 \\ 0 \\ 0.40 \end{bmatrix}$$

Sumando,

$$\beta_{\min} = \begin{bmatrix} 1.0 \\ 0.25 \\ 1.0 \\ 0.40 \end{bmatrix}$$

Las filas cuyo error era positivo reciben el nuevo valor de

$$\beta.$$

Las demás permanecen sin cambios.

Actualización vectorizada de $\beta_{\max}$

La actualización del límite superior es completamente análoga.

```
$beta_max =
    ($is_less * $beta)
  + ($is_greater * $beta_max);
```

Supongamos

$$\beta_{\max} = \begin{bmatrix} 10 \\ 10 \\ 10 \\ 10 \end{bmatrix}$$

Utilizando las mismas máscaras,

obtenemos

$$is_less \cdot \beta = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 1 \end{bmatrix}$$

y

$$is_greater \cdot \beta_{\max} = \begin{bmatrix} 10 \\ 0 \\ 10 \\ 0 \end{bmatrix}$$

Finalmente,

$$\beta_{\max} = \begin{bmatrix} 10 \\ 1 \\ 10 \\ 1 \end{bmatrix}$$

Ahora,

las observaciones cuyo error era negativo almacenan el valor actual de

$$\beta$$

como nuevo límite superior.

Las máscaras como interruptores algebraicos

Una máscara formada por ceros y unos puede interpretarse como un interruptor.

Si

$$m = 1,$$

el término permanece activo.

Si

$$m = 0,$$

el término desaparece.

Por ejemplo,

$$m \cdot x = \begin{cases} x, & m = 1 \\ 0, & m = 0 \end{cases}$$

Esto permite escribir una selección condicional como una única expresión algebraica.

En lugar de

si condición

A

si no

B

utilizamos

$$mA + (1 - m)B$$

La implementación de MXNet aplica esta idea de manera simultánea sobre miles de observaciones.

Cada fila activa únicamente el término que le corresponde,

sin utilizar estructuras condicionales.

Ventajas

- No existen bucles adicionales.
- No existen instrucciones `if`.
- Todas las observaciones se actualizan en paralelo.
- La implementación aprovecha al máximo el paralelismo del hardware.

Este principio de **selección mediante máscaras** aparece con frecuencia en algoritmos de aprendizaje profundo y constituye una de las técnicas fundamentales de la programación tensorial.

Las cuatro posibilidades de una búsqueda binaria

Después de actualizar los límites

$$\beta_{\min} \quad \text{y} \quad \beta_{\max},$$

debemos calcular el siguiente valor de

$$\beta.$$

En una búsqueda binaria clásica existen únicamente cuatro posibilidades.

Caso 1

La entropía es demasiado grande

y todavía no existe un límite superior.

$$\beta_{\max} = +\infty$$

Entonces,

duplicamos el valor actual.

$$\beta \leftarrow 2\beta$$

Caso 2

La entropía es demasiado grande

y ya existe un límite superior.

Calculamos el punto medio.

$$\beta \leftarrow \frac{\beta + \beta_{\max}}{2}$$

Caso 3

La entropía es demasiado pequeña

y todavía no existe un límite inferior.

Reducimos el valor actual a la mitad.

$$\beta \leftarrow \frac{\beta}{2}$$

Caso 4

La entropía es demasiado pequeña

y ya existe un límite inferior.

Calculamos el punto medio.

$$\beta \leftarrow \frac{\beta + \beta_{\min}}{2}$$

Estas cuatro posibilidades abarcan todos los casos posibles.

Construcción de los cuatro candidatos

La implementación calcula previamente los cuatro posibles valores.

```
my $b_greater_max_init =  
  $beta * 2.0;  
  
my $b_greater_not_init =  
  ($beta + $beta_max) / 2.0;  
  
my $b_less_min_init =  
  $beta / 2.0;  
  
my $b_less_not_init =  
  ($beta + $beta_min) / 2.0;
```

Cada vector representa una posible actualización de

β .

Variable	Significado
<code>b_greater_max_init</code>	Duplicar β
<code>b_greater_not_init</code>	Punto medio con $\beta_{\max}$
<code>b_less_min_init</code>	Dividir β entre dos
<code>b_less_not_init</code>	Punto medio con $\beta_{\min}$

En este momento,

el algoritmo todavía no sabe cuál de los cuatro utilizará para cada observación.

La selección se realizará mediante máscaras tensoriales.

Selección completamente vectorizada

Finalmente,

la implementación calcula el nuevo vector

```
$beta =
    ($is_greater * $mask_max_init * $b_greater_max_init)
  + ($is_greater * (1 - $mask_max_init) * $b_greater_not_init)
  + ($is_less * $mask_min_init * $b_less_min_init)
  + ($is_less * (1 - $mask_min_init) * $b_less_not_init);
```

Cada término representa exactamente uno de los cuatro casos estudiados.

Condición	Valor seleccionado
$\Delta H > 0$ y no existe $\beta_{\max}$	2β
$\Delta H > 0$ y existe $\beta_{\max}$	$\frac{\beta + \beta_{\max}}{2}$
$\Delta H < 0$ y no existe $\beta_{\min}$	$\frac{\beta}{2}$
$\Delta H < 0$ y existe $\beta_{\min}$	$\frac{\beta + \beta_{\min}}{2}$

Las máscaras contienen únicamente valores

0 o 1,

por lo que únicamente uno de los cuatro términos permanece activo para cada observación.

Los otros tres se anulan automáticamente.

Resumen completo del algoritmo `_binary_search_perplexity`

El procedimiento completo puede resumirse en los siguientes pasos.

Paso 1

Calcular las probabilidades Gaussianas.

```
$row_P =
    nd->exp(-$sqdistances_f64 * $beta);
```

Paso 2

Normalizar cada fila.


```
$row_P_normalized =  
  $row_P / $sum_Pi;
```

Paso 3

Calcular la entropía.

```
$entropy =  
  nd->log($sum_Pi)  
  + $beta * $sum_disti_Pi;
```

Paso 4

Comparar con la entropía deseada.

```
$entropy_diff =  
  $entropy  
  - $desired_entropy;
```

Paso 5

Actualizar los límites.

\$beta_min

\$beta_max

Paso 6

Calcular el nuevo valor de

β .

Paso 7

Repetir el proceso hasta completar las iteraciones de la búsqueda binaria.

```
for my $step (0 .. $n_steps - 1)
```

Resultado

El algoritmo devuelve

```
return $row_P_normalized;
```

es decir,

la matriz de probabilidades condicionales

$$P_{j|i}$$

que será utilizada por la siguiente etapa del algoritmo t-SNE para construir la matriz de probabilidades simétrica y, posteriormente, minimizar la divergencia de Kullback-Leibler.

Ejercicio 2

Primera iteración de la búsqueda binaria

Considere una búsqueda binaria con

$$\beta = \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix}$$

$$\beta_{\min} = \begin{bmatrix} -10^{20} \\ -10^{20} \\ -10^{20} \\ -10^{20} \end{bmatrix}$$

$$\beta_{\max} = \begin{bmatrix} 10^{20} \\ 10^{20} \\ 10^{20} \\ 10^{20} \end{bmatrix}$$

Después de calcular la entropía se obtiene

$$\Delta H = \begin{bmatrix} 0.40 \\ -0.20 \\ 0.15 \\ -0.35 \end{bmatrix}$$

Calcule:

1. La máscara

is_greater

2. La máscara

is_less

3. El nuevo valor de

$\beta_{\min}$

4. El nuevo valor de

$\beta_{\max}$

5. El nuevo vector

β

considerando que todavía no existen límites previos.

Solución del Ejercicio 2

Paso 1

Construimos

$$is_greater = (\Delta H > 0)$$

$$\begin{bmatrix} 1 \\ 0 \\ 1 \\ 0 \end{bmatrix}$$

Paso 2

La máscara complementaria es

$$is_less = 1 - is_greater$$

$$\begin{bmatrix} 0 \\ 1 \\ 0 \\ 1 \end{bmatrix}$$

Paso 3

Actualizamos

$$\beta_{\min} = is_greater \cdot \beta + is_less \cdot \beta_{\min}$$

obteniendo

$$\beta_{\min} = \begin{bmatrix} 1 \\ -10^{20} \\ 1 \\ -10^{20} \end{bmatrix}$$

Paso 4

Actualizamos

$$\beta_{\max} = is_less \cdot \beta + is_greater \cdot \beta_{\max}$$

obteniendo

$$\beta_{\max} = \begin{bmatrix} 10^{20} \\ 1 \\ 10^{20} \\ 1 \end{bmatrix}$$

Paso 5

Como todavía no existen límites previos,

las cuatro posibilidades se reducen a dos.

Si

$$\Delta H > 0,$$

duplicamos

$$\beta.$$

Si

$\Delta H < 0,$

dividimos

β

entre dos.

Por tanto,

$$\beta = \begin{bmatrix} 2 \\ 0.5 \\ 2 \\ 0.5 \end{bmatrix}$$

Resultado final

Variable	Valor
<i>is_greater</i>	$\begin{bmatrix} 1 \\ 0 \\ 1 \\ 0 \end{bmatrix}$
<i>is_less</i>	$\begin{bmatrix} 0 \\ 1 \\ 0 \\ 1 \end{bmatrix}$
$\beta_{\min}$	$\begin{bmatrix} 1 \\ -10^{20} \\ 1 \\ -10^{20} \end{bmatrix}$
$\beta_{\max}$	$\begin{bmatrix} 10^{20} \\ 1 \\ 10^{20} \\ 1 \end{bmatrix}$
Nuevo β	$\begin{bmatrix} 2 \\ 0.5 \\ 2 \\ 0.5 \end{bmatrix}$

```
In [1]: use strict;
use warnings;
use Data::Dump qw(dump);
use AI::MXNet qw(mx nd);
```

```
In [25]: # Binary search for sigmas of conditional Gaussians.
# sklearn/manifold/_utils.pyx
# This approximation reduces the computational complexity from O(N^2) to
# O(uN).
#
# Parameters
#-----
# sqdistances : ndarray of shape (n_samples, n_neighbors), dtype=np.float32
#   Distances between training samples and their k nearest neighbors.
#   When using the exact method, this is a square (n_samples, n_samples)
#   distance matrix. The TSNE default metric is "euclidean" which is
#   interpreted as squared euclidean distance.
#
# desired_perplexity : float
#   Desired perplexity (2^entropy) of the conditional Gaussians.
#
# verbose : int
```

```

# Verbosity level.
#
# Returns
#-----
# P : ndarray of shape (n_samples, n_samples), dtype=np.float64
# Probabilities of conditional Gaussian distributions p_i|j.

# Fully Vectorized _binary_search_perplexity
# sklearn/manifold/_utils.pyx
sub _binary_search_perplexity_mxnet {
    my ($sqdistances, $desired_perplexity, $verbose) = @_;

    my ($n_samples, $n_neighbors) = @{$sqdistances->shape};

    my $n_steps = 100;
    my $EPSILON_DBL = 1e-8;
    my $PERPLEXITY_TOLERANCE = 1e-5;
    my $desired_entropy = log($desired_perplexity);

    # Parámetros internos en float64 (equivalente a cdef double beta)
    my $beta = nd->ones([$n_samples, 1], dtype => 'float64');
    my $beta_min = nd->ones([$n_samples, 1], dtype => 'float64') * -1e20;
    my $beta_max = nd->ones([$n_samples, 1], dtype => 'float64') * 1e20;

    my $diag_mask;
    if ($n_neighbors == $n_samples) {
        $diag_mask = nd->ones([$n_samples, $n_neighbors], dtype => 'float64') - nd->eye($n_sample
    }

    my $sqdistances_f64 = $sqdistances->astype('float64');
    my $row_P_normalized;

    # Global binary search loop
    for my $step (0 .. $n_steps - 1) {

        # === AJUSTE DE SUBFLUJO: Prevenir que MXNet trunque prematuramente a 0.0 ===
        my $exponent = -$sqdistances_f64 * $beta;

        # -708.39 es el límite inferior exacto para float64 donde math.exp se vuelve 0.0
        # Usamos el método .maximum() de MXNet para asegurar consistencia de dtypes
        my $floor_tensor = nd->ones($exponent->shape, dtype => 'float64') * -708.3964185322641;
        $exponent = $exponent->maximum($floor_tensor);

        my $row_P = nd->exp($exponent);

        if (defined $diag_mask) {
            $row_P = $row_P * $diag_mask; # float64 * float64
        }

        # 2. Compute sum across rows
        my $sum_Pi = nd->sum($row_P, axis => 1, keepdims => 1)->maximum($EPSILON_DBL);

        # 3. Normalize (Se mantiene en float64)
        $row_P_normalized = $row_P / $sum_Pi;

        # 4. Compute sum_disti_Pi (Operar sqdistances_f64 para mantener homogeneidad con row_P_no
        my $sum_disti_Pi = nd->sum($sqdistances_f64 * $row_P_normalized, axis => 1, keepdims => 1

        # 5. Compute Entropy globally
        my $entropy = nd->log($sum_Pi) + $beta * $sum_disti_Pi;
        my $entropy_diff = $entropy - $desired_entropy;

        my $is_greater = ($entropy_diff > 0.0)->astype('float64');
        my $is_less = nd->ones([$n_samples, 1], dtype => 'float64') - $is_greater;

        my $mask_max_init = ($beta_max >= 1e19)->astype('float64');
        my $mask_min_init = ($beta_min <= -1e19)->astype('float64');

        # Actualizaciones de límites estables
        $beta_min = ($is_greater * $beta) + ($is_less * $beta_min);
        $beta_max = ($is_less * $beta) + ($is_greater * $beta_max);

        # Posibles caminos matemáticos (Todos en float64 inherente)
        my $b_greater_max_init = $beta * 2.0;
        my $b_greater_not_init = ($beta + $beta_max) / 2.0;

```

```

my $b_less_min_init = $beta / 2.0;
my $b_less_not_init = ($beta + $beta_min) / 2.0;

# Construcción limpia del vector de Betas usando álgebra de conmutación 100% float64
$beta = ($is_greater * $mask_max_init * $b_greater_max_init) +
        ($is_greater * (nd->ones([n_samples, 1], dtype => 'float64') - $mask_max_init) *
        ($is_less * $mask_min_init * $b_less_min_init) +
        ($is_less * (nd->ones([n_samples, 1], dtype => 'float64') - $mask_min_init) * $b

}

# Retorna P_normalized conservando su estructura float64 final
return $row_P_normalized;
}

```

Subroutine _binary_search_perplexity_mxnet redefined at reply input line 27.

```

In [26]: sub pdist_sq {
my ($X) = @_;

$X = $X->astype('float64');
my $n_samples = $X->shape->[0];

# ||x_i - x_j||^2 = x_i^2 - 2*x_i*x_j^T + x_j^2 (Matriz completa intermedia)
my $X_sum_squares = nd->sum($X * $X, axis => 1, keepdims => 1);
my $X_inner_prod = nd->dot($X, $X->T);
my $sqdistances = $X_sum_squares - (2 * $X_inner_prod) + $X_sum_squares->T;
$distances = nd->clip($sqdistances, a_min => 0.0, a_max => 'Inf');

return $sqdistances;
}

```

Subroutine pdist_sq redefined at reply input line 1.

```

In [27]: # 1. X dataset
my $X_data = nd->array([
    [1.0, 0.1, -0.2, 0.5],
    [1.1, 0.2, -0.1, 0.4],
    [0.0, 5.2, 4.1, -3.0],
    [0.1, 5.0, 4.3, -3.1],
    [2.0, -1.0, 0.5, 1.2]
], dtype => 'float64');

```

Out[27]: <AI::MXNet::NDArray 5x4 @cpu(0)>

```

In [28]: # 2. Calculamos las distancias condensadas
my $distances = pdist_sq($X_data);
nd->printf("%.2f", $distances);

```

```

[[ 0.00  0.04 57.75 58.03  3.19]
 [ 0.04  0.00 55.41 55.65  3.25]
 [57.75 55.41  0.00  0.10 73.04]
 [58.03 55.65  0.10  0.00 72.54]
 [ 3.19  3.25 73.04 72.54  0.00]]

```

<AI::MXNet::NDArray 5x5 @cpu(0)> AI::MXNet::NDArray float64

Out[28]: 1

```

In [29]: # Binary search for sigmas of conditional Gaussians.
my $prob_matrix = _binary_search_perplexity_mxnet($distances, 2, 0);
nd->printf("Probability Matrix:\n%.2f\n", $prob_matrix);
nd->printf("Sum of its columns:\n%.2f", nd->sum($prob_matrix, axis => 1));

```

```

Probability Matrix:
[[0.00 0.58 0.00 0.00 0.41]
 [0.59 0.00 0.00 0.00 0.41]
 [0.07 0.08 0.00 0.81 0.04]
 [0.07 0.08 0.81 0.00 0.04]
 [0.50 0.50 0.00 0.00 0.00]]

```

<AI::MXNet::NDArray 5x5 @cpu(0)> AI::MXNet::NDArray float64

```

Sum of its columns:
[1.00 1.00 1.00 1.00 1.00]

```

<AI::MXNet::NDArray 5 @cpu(0)> AI::MXNet::NDArray float64

Out[29]: 1

```

In [24]: sub _binary_search_perplexity_scalar {
    my ($sqdistances, $desired_perplexity, $verbose) = @_;

    # Forzar float32 en la entrada para que coincida con sklearn
    # $sqdistances = $sqdistances->astype('float32');
    my ($n_samples, $n_neighbors) = @{$sqdistances->shape};
    my $using_neighbors = $n_neighbors < $n_samples;

    my $n_steps = 100;
    my $EPSILON_DBL = 1e-8;
    my $PERPLEXITY_TOLERANCE = 1e-5;
    my $desired_entropy = log($desired_perplexity);

    # Matriz final P en float64
    my $P = nd->zeros([$n_samples, $n_neighbors], dtype => 'float64', ctx => $sqdistances->cont
    my $beta_sum = 0.0;

    for my $i (0 .. $n_samples-1) {
        my $beta = 1.0;
        my $beta_min = -9e999; # -inf
        my $beta_max = 9e999; # +inf

        # Extraemos la fila actual a un array nativo de Perl para evitar
        # el ruido de precisión de los operadores de MXNet
        my $row_data = $sqdistances->slice($i)->asarray();

        # Array temporal de Perl para la fila P_i actual
        my $Pi_row = [(0.0) x $n_neighbors];

        for (my $step = 0; $step < $n_steps; $step++) {
            my $sum_Pi = 0.0;

            # 1. Calcular Exponenciales y acumular la suma secuencialmente (Idéntico a Cython)
            for my $j (0 .. $n_neighbors-1) {
                if ($j != $i || $using_neighbors) {
                    $Pi_row->[$j] = exp(-$row_data->[$j] * $beta);
                    $sum_Pi += $Pi_row->[$j];
                } else {
                    $Pi_row->[$j] = 0.0;
                }
            }

            $sum_Pi = $EPSILON_DBL if $sum_Pi == 0.0;

            # 2. Normalizar y calcular sum_disti_Pi en el mismo orden secuencial
            my $sum_disti_Pi = 0.0;
            for my $j (0 .. $n_neighbors-1) {
                $Pi_row->[$j] /= $sum_Pi;
                $sum_disti_Pi += $row_data->[$j] * $Pi_row->[$j];
            }

            my $entropy = log($sum_Pi) + $beta * $sum_disti_Pi;
            my $entropy_diff = $entropy - $desired_entropy;

            # 3. Guardar la fila calculada en la matriz P de MXNet
            # Lo hacemos aquí para asegurar que conserve el último estado de la búsqueda
            my $Pi_nd = nd->array($Pi_row, dtype => 'float64', ctx => $sqdistances->context);
            $P->slice($i, ':')->set($Pi_nd);

            # Criterio de parada
            last if abs($entropy_diff) <= $PERPLEXITY_TOLERANCE;

            # Actualización de Beta (Búsqueda Binaria)
            if ($entropy_diff > 0.0) {
                $beta_min = $beta;
                $beta = $beta_max > 1e300 ? $beta * 2.0 : ($beta + $beta_max) / 2.0;
            } else {
                $beta_max = $beta;
                $beta = $beta_min < -1e300 ? $beta / 2.0 : ($beta + $beta_min) / 2.0;
            }
        }
        $beta_sum += $beta;

        if ($verbose && (((i + 1) % 1000) == 0 || (i + 1) == $n_samples)) {

```

```

        print "[t-SNE] Computed conditional probabilities for sample ", $i + 1, " / ", $n_sampl
    }
}

if ($verbose) {
    printf "[t-SNE] Mean sigma: %f\n", sqrt($n_samples / $beta_sum);
}

return $P;
}

```

Subroutine _binary_search_perplexity_scalar redefined at reply input line 1.

```

In [8]: $prob_matrix = _binary_search_perplexity_scalar($distances, 2, 0);
nd->printf("Probability Matrix:\n%.2f\n", $prob_matrix);
nd->printf("Sum of its columns:\n%.2f", nd->sum($prob_matrix, axis => 1));

```

Probability Matrix:

```

[[0.00 0.58 0.00 0.00 0.41]
 [0.59 0.00 0.00 0.00 0.41]
 [0.07 0.08 0.00 0.81 0.04]
 [0.07 0.08 0.81 0.00 0.04]
 [0.50 0.50 0.00 0.00 0.00]]

```

<AI::MXNet::NDArray 5x5 @cpu(0)> AI::MXNet::NDArray float64

Sum of its columns:

```

[1.00 1.00 1.00 1.00 1.00]

```

<AI::MXNet::NDArray 5 @cpu(0)> AI::MXNet::NDArray float64

Out[8]: 1